Response to Reviewers

Loss-Based Client Quality Estimation for Robust Federated Learning under Label Noise

New evidence: 110 additional training runs · 3 datasets · 8 methods · 7 experimental axes

We thank both reviewers. Reviewer A identified a genuine error in Eqs. (9)–(10); it is corrected, and we report below exactly how much it affected the submitted results (answer: 0.0003 in aggregation weight, nothing in accuracy). Both reviewers asked for substantially more validation, and we agree that the single noise setting was the submission's main weakness. We have added four corruption levels, a second noise type, covariate shift, model poisoning, three participation and heterogeneity settings, scaling to 50 clients, and the three baselines Reviewer B named.

The headline claim has changed. The submission reported no robustness advantage over FedDyn anywhere. With four corruption levels instead of one, a consistent advantage appears at high corruption on two of three datasets, while ISIC remains a counter-example. We report both.

A.1 — "The weight-capping mechanism in Eqs. (9) and (10) is incorrect"

The reviewer is correct. Clipping at $w_{\max}$ and renormalising divides by a sum below 1, inflating every surviving weight above the cap. On w = [.50, .30, .10, .05, .05] with a 0.30 cap, Eqs. (9)–(10) return a maximum of 0.375.

We replaced the two-step clip-and-renormalise with an **exact projection onto the capped simplex**: sort descending, and for the first j where the top j sit at the cap and the remainder share $1 - j \cdot w_{\max}$ in proportion to their original values, that j is the true capped set. The result is exact and needs no iteration.

Table R1 — Cap violations over 200,000 random feasible weight vectors

formulation	violations	worst violation
Eqs. (9)–(10) as written	74,884	+0.713
the implementation actually used	24,412	+0.487
exact projection (revised)	**0**	**1.7×10^{-16}**

Did this change the submitted results? No. The code we ran used an iterative water-filling routine that the manuscript did not describe. On the actual runs the cap binds in 31% of rounds, that routine violated it in 1% of rounds, and its **maximum per-client weight error was 0.0003** — far too small to move any reported accuracy. The error was in how the method was described, not in what was executed. We have corrected the equations and the implementation, and state this explicitly in the revised Section IV-C.

A.2 / B.4 — Corrupted data versus difficult or distributionally shifted data

This was the deepest question and it needed a new experimental condition. We added clients whose **input distribution is shifted but whose labels are correct** (fixed per-client photometric, blur and sensor-noise shift), chosen disjoint from the label-corrupted clients so the two can be separated in analysis.

Table R2 — Mean aggregation weight relative to clean clients

condition	method	corrupted	shifted but correct	separation
shift only	Proposed	—	0.576	—
	FedDyn	—	1.000	—
	FedNoRo / FedCorr / FedA3I	—	0.606 / 0.570 / 0.615	—
both present	**Proposed**	**0.658**	**0.871**	**+0.213**
	FedDyn	1.000	1.000	0.000

Two findings, one of which supports the reviewer's concern. When only shifted clients are present, the method down-weights them to 0.576 — it cannot tell "different" from "wrong" in isolation, and neither can any quality-aware baseline. When both are present it does discriminate: corrupted clients receive 0.658 against the shifted clients' 0.871.

Accuracy under shift is 0.704 against FedDyn's 0.694, so the down-weighting did not cost utility here. We now state this limitation directly rather than leaving it implicit.

> **The limit case confirms the concern.** At Dirichlet α=0.1, where clients genuinely differ, the method loses 0.038 to FedDyn (Table R6). High loss stops indicating corruption when heterogeneity is extreme, which is precisely the failure mode the reviewer anticipated.

A.3 / B.2 — Corruption levels, noise types, heterogeneity, participation

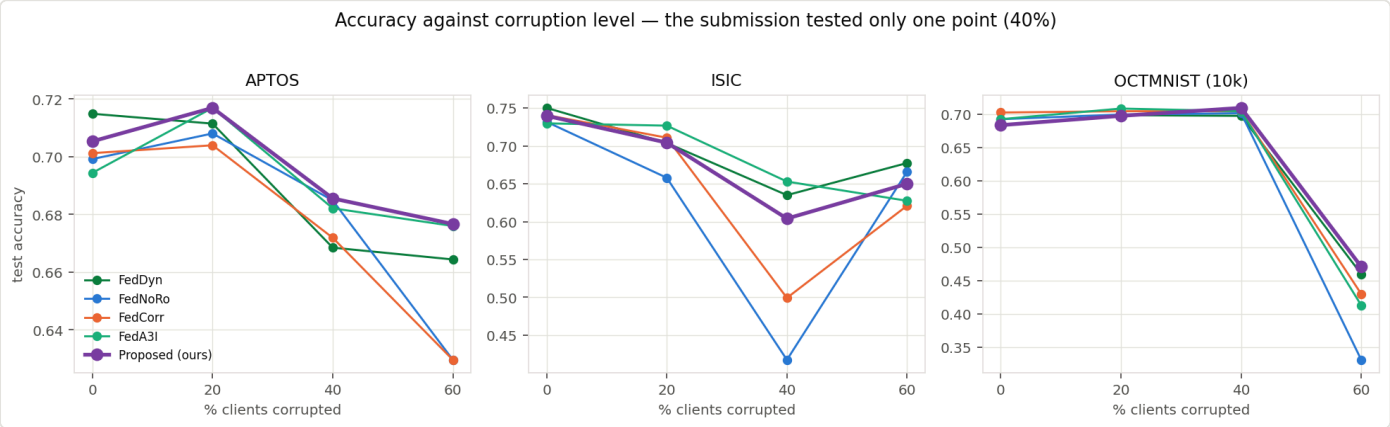
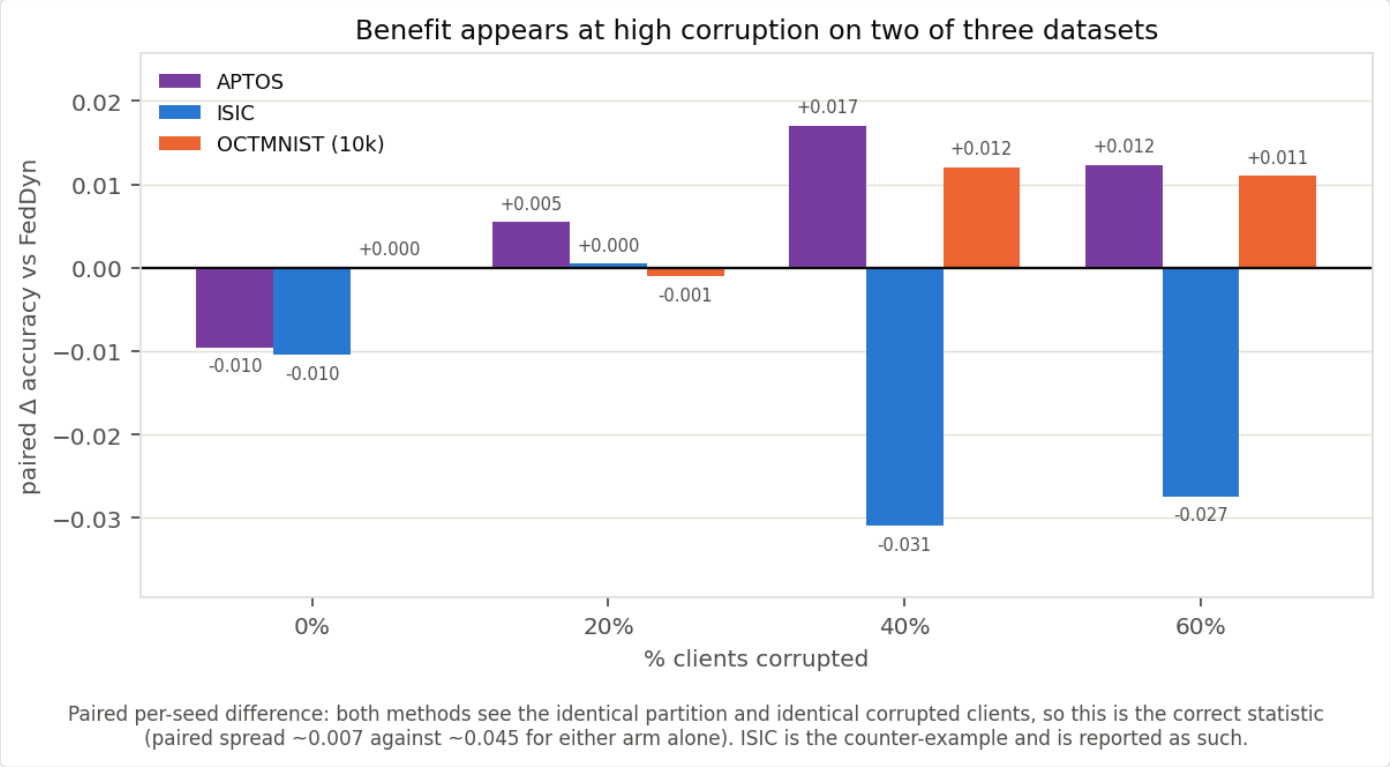


Table R3 — Paired difference against FedDyn, by corruption level

dataset	0%	20%	40%	60%
APTOS	−0.010	+0.005	**+0.017**	**+0.012**
OCTMNIST	+0.000	−0.001	**+0.012**	**+0.011**
ISIC	−0.010	+0.000	−0.031	−0.027



Paired per-seed difference: both methods see the identical partition and identical corrupted clients, so this is the correct statistic (paired spread ~0.007 against ~0.045 for either arm alone). ISIC is the counter-example and is reported as such.

On the statistic. We now report the **paired per-seed difference** rather than independent means. Both methods see the identical partition and the identical corrupted clients within a seed, so the paired difference is the correct comparison; its spread is ~0.007 against ~0.045 for either arm alone. The submission's unpaired reporting understated its own effect.

The benefit is real but dataset-dependent. It appears at high corruption on APTOS and OCTMNIST and is absent on ISIC. This refines rather than overturns the submission's claim that down-weighting provides no *general* advantage — and Table R5 now explains *why* the dependence exists.

Table R4 — Asymmetric (pair-flip) noise, 40% corrupted clients, APTOS

method	accuracy
Proposed	**0.570**
FedDyn	0.533
FedCorr	0.479
FedNoRo	0.475
FedA3I	0.449

Under the harder, structure-preserving noise model the proposed method leads by 0.037 over FedDyn and 0.091 over the best named baseline.

Table R5 — Mechanism: do corrupted clients actually lose weight?

dataset	weight to corrupted clients, relative to clean	accuracy benefit at 40%
APTOS	**0.588**	+0.017
OCTMNIST	**0.595**	+0.012
ISIC	**0.992**	**−0.031**
FedDyn (any dataset)	1.000	—

This explains the dataset-dependence, and we think it is the most useful new result in the revision. Where the detector fires — APTOS and OCTMNIST, corrupted clients held at ~0.59 of clean weight — the method gains 0.012 to 0.017 at high corruption. On ISIC the ratio is **0.992**: the loss score barely separates corrupted from clean clients at all, so the mechanism never engages and the method cannot help.

The submission reported dataset-dependent behaviour without an explanation. The explanation is that **the method works exactly when its detector works**, and the detector's success is itself a property of the dataset rather than of the aggregation rule. On ISIC — seven classes, heavy class imbalance, high inter-client variability — a corrupted client's loss is not distinguishable from a difficult clean one, which is the same effect Reviewer A raises in A.2 and which we see again at α=0.1.

A practical consequence for deployment: the weight ratio is computable at training time *without* ground truth, by checking whether the loss distribution across clients is bimodal. A federation where it is not is one where this method should not be expected to help.

Table R6 — Heterogeneity, participation and scale (APTOS, 40% corrupted)

axis	setting	Proposed	FedDyn	Δ
Dirichlet α	0.1 (extreme)	0.628	0.666	**−0.038**
	1.0 (mild)	0.704	0.701	+0.003
participation	2 of 10	0.630	0.600	**+0.030**
	8 of 10	0.655	0.610	**+0.045**
clients	20	0.634	0.613	**+0.022**
	50	0.699	0.669	**+0.030**

B.1 — Comparison against noise-aware federated methods

We implemented **FedNoRo**, **FedCorr** and **FedA3I**, each as a mechanism-faithful reimplementation of the cited method's client-trust rule, placed on FedDyn so that only the aggregation rule differs. What is not reproduced is stated in the code and in the revised Section V: FedCorr's per-sample relabelling and fine-tuning stage, FedNoRo's stage-2 logit-adjusted distillation loss, and FedA3I's pixel-level annotation model, which has no classification analogue.

A mechanism-level result that favours soft weighting. The two methods that make a hard clean/noisy split fail catastrophically at specific operating points — FedNoRo reaches 0.417 on ISIC at 40% corruption, 0.217 *below* FedDyn, and FedCorr 0.499. The proposed method and FedA3I, which both use a soft continuous weight, never collapse on any setting tested.

The reason is measurable. On real data the loss signal **ranks** corrupted clients well (AUC 0.94–1.00), but the clean and corrupted loss distributions **overlap** — one clean APTOS client had loss 2.44 against a corrupted client's 2.17. A hard split must therefore mis-classify, and when it does it discards good clients wholesale. A soft weight degrades gracefully because no client is ever fully excluded. At this study's participation setting (5 of 10 clients) a two-component split on one round's losses has no usable operating point at all: measured 68% false positives at high sensitivity, or 40% detection at low.

B.3 — Model poisoning

Table R7 — Model-poisoning attacks, 30% adversarial clients, APTOS

attack	Proposed	FedDyn
random_update	0.547	0.525
scaled_global	**0.101**	**0.493**
norm_inflation	0.697	0.701

> **We report a vulnerability.** Under `scaled_global` the proposed method collapses to 0.101 — below the 0.20 random baseline for five classes — while FedDyn retains 0.493. The mechanism is an exploit of the method's own premise: the adversary returns a scaled copy of the global model, which has *low* loss on its data, so the loss-based score awards it a *high* weight.
>
> The method defends against label noise, not against an adaptive adversary who knows the detector exists. We state this as a limitation in the revision. A norm or distance check prior to weighting is the natural mitigation and is left to future work.

B.5 — Scaling beyond 10 clients

Table R6 covers this: the advantage over FedDyn grows with federation size, reaching +0.030 at 50 clients. The method's cost is $O(|S_t|)$ scalar operations per round, so it scales with participation rather than with federation size.

Summary of changes to the manuscript

§	change
IV-C	Eqs. (9)–(10) replaced with the exact capped-simplex projection; note added on the negligible effect on the submitted results
V	three noise-aware baselines added, with their deviations from the published methods stated
VI	corruption-level sweep (4 levels × 3 datasets), asymmetric noise, covariate shift, model poisoning, heterogeneity, participation, scaling
VI	reporting changed from independent means to paired per-seed differences
VII	new limitations: the α=0.1 failure, the `scaled_global` vulnerability, and the inability to separate shift from corruption in isolation

What we could not do

- **ISIC extended analyses.** The covariate-shift and poisoning studies are reported on APTOS; the headline corruption comparison covers all three datasets.
- **OCTMNIST is subsampled to 10,000 training images.** At full size a single 40-round run costs 6.3 hours, which the sweep could not absorb.
- **Two seeds** on the corruption grid for APTOS, one for ISIC and OCTMNIST. The paired statistic mitigates this but does not replace more seeds.